```
makeServer = ->
  responseData = ''
  server = http.createServer (request, response) ->
    response.writeHead 200, 'Content-Type': 'text/html'
    response.end JSON.stringify responseData
  server.listen 8888, '127.0.0.1'
  (data) ->
    responseData = data

main = (fileName) ->
  server = makeServer()

  loadData = ->
    console.log 'Loading data'
    sortedCompetitorsFromFile fileName, (data) ->
      console.log 'Data loaded'
      server data

  loadData()
  fs.watchFile fileName, loadData

if process.argv[2]
  main process.argv[2]
  console.log "starting server on port 8888"
else
  console.log "usage: coffee 9.1.coffee [file]"
```

Uses closure to keep data

loadData is invoked immediately and passed to file watcher

Deals with program arguments

Running the application with the competitors file containing 1,500 competitors serves the resulting JSON at `http://localhost:8888/`:

```
> coffee listing.91.coffee competitors.txt
Loading data
Starting server on port 8888
Data loaded
```

If you visit this page in a web browser, you'll see that the performance is fine. Nothing to worry about; the array is being sorted synchronously but it's not hurting the program—not yet anyway. It's tempting early on to assume that you only need to think asynchronous when reading things like files and network resources, but you soon find out that this is wrong. Read on.

9.1.3. Performance